

Automatic News Headline Generation

NLP-Assignment - 2 GS-2
Group 116

Group Details

Name	BITS ID	Contribution %
SUJEET KUMAR YADAV	2025AA05326	100%
SHELAR SACHIN KRISHNA	2025AA05387	100%
POWAR SAGAR GANPATI	2025AA05421	100%
PRASAD SHIVAJI KULKARNI	2025AA05444	100%
PATTIPATI SAI CHANDAN SINGH	2025AB05149	100%

1. Problem Analysis

1.1 Application Domain and Stakeholders

The application domain is automated content curation for digital news publishing. News desks, wire services and content-aggregation platforms publish a large volume of articles every day, and each article needs a short, accurate and attention-holding headline before it can be indexed, displayed or shared.

Primary stakeholders:

- News desk editors and sub-editors, who currently write headlines manually and could use generated headlines as a first draft.
- Content aggregation / news-recommendation platforms, which need consistent headlines for articles ingested from many sources.
- Readers, whose engagement and comprehension depend on headline clarity and accuracy.
- Search and social platforms, where headline quality affects click-through and search visibility.

1.2 Problem Statement and Functional Requirements

Given the body text of a news article, the system must automatically generate a single-line, abstractive headline that summarises the article's main point in natural language, without being restricted to copying exact spans of the source text.

Functional requirements:

- Accept raw article text (pasted or uploaded) as input.
- Clean and tokenize the input consistently with the training-time preprocessing.
- Generate a headline using an encoder–decoder model, using either greedy or beam-search decoding.
- Optionally return multiple candidate headlines (top-k) for the same article.
- Expose the functionality through a web application usable without programming knowledge.

Headline generation vs. extractive summarization:

Extractive summarization selects and concatenates existing sentences or phrases directly from the source article, so every word in the summary is guaranteed to appear in the input.

Abstractive headline generation, which this project implements, instead uses an encoder–decoder model to produce new phrasing that may paraphrase, compress or reorder the article's content — similar to how a human sub-editor writes a headline rather than quoting a sentence verbatim. This allows much shorter, punchier output than extractive methods can produce, at the cost of a harder learning problem and a risk of factual drift if the model is not well trained.

1.3 Expected Input and Output

Input: the plain-text body of a news article, expected to be roughly 50–400 words (the assignment's target range). The application also accepts batches of articles via .txt (blank-line separated) or .csv (one article per row) upload.

Output: a single-line abstractive headline, targeted at roughly 5–15 words, matching the length distribution of real headlines in the training corpus (mean 9.6 words, median 10 words — see §2.1). The application additionally shows up to top-k beam-search candidate headlines per article.

2. Data Collection and Preprocessing

2.1 Dataset

Name: News Summary Dataset (a.k.a. "Inshorts News Data").

Source: Kaggle — <https://www.kaggle.com/datasets/sunnysai12345/news-summary> (mirrored, and used directly in this project, at https://github.com/sunnysai12345/News_Summary).

Size used: 98,401 raw article–headline pairs from news_summary_more.csv; 98,373 pairs remain after cleaning, within the assignment's 20,000–200,000 record guidance. To keep training time reasonable on CPU-only hardware, a 6,000 / 800 / 800 train/validation/test subset was sampled for the model actually trained in this submission. The full processed split (88,535 / 4,918 / 4,920) is also produced by the same pipeline and can be trained on directly given GPU access.

Statistic (full cleaned corpus)	Value
Total cleaned records	98373
Article words (mean / median)	58.2 / 59
Headline words (mean / median)	9.6 / 10
Vocabulary size (analysis)	30004

A handful of longer, real full-length articles (news_summary.csv, ctext column, median 283 words) are kept separately as realistic demo inputs for the application, covering the upper end of the assignment's target article-length range.

2.2 Text Preprocessing

Implemented in data/prepare_data.py:

- HTML/boilerplate removal (residual tags, URLs, "Read more"/"Also read" style boilerplate).
- Unicode normalisation (NFKC), smart-quote/dash normalisation, and whitespace collapsing.
- Lowercasing for the normalised view used in vocabulary construction.
- Removal of empty, duplicate, and very short/long records (article and headline word-count bounds).
- Addition of explicit <sos> / <eos> tokens around each target headline.

2.3 Dataset Preparation for Training

- Word-level vocabulary built from training data only (8,000 tokens, min frequency 2), with <pad>, <unk>, <sos>, <eos> special tokens.
- Articles truncated/padded to a fixed maximum source length (100 tokens); headlines to a fixed maximum target length (20 tokens).
- Deterministic (seeded) train / validation / test split, produced once by prepare_data.py and reused by make_train_subset.py to build the 6,000/800/800 CPU-training subset.

3. Model Development

3.1 Architecture

The headline generator is a from-scratch Transformer encoder–decoder (training/model.py), built on torch.nn.Transformer following Vaswani et al., "Attention Is All You Need" (2017). The encoder uses multi-head self-attention over the padded, positionally-encoded article tokens; the decoder uses masked (causal) self-attention combined with encoder–decoder cross-attention, so every predicted headline token attends over the full encoded article. Sinusoidal positional encoding is used, and encoder padding is masked out with a key-padding mask.

Hyperparameter	Value
Embedding size (d_model)	256
Attention heads	4
Encoder layers	3
Decoder layers	3
Feed-forward size	512
Dropout	0.1
Vocabulary size	8000
Max source / target length	100 / 20
Batch size	24
Optimizer / learning rate	Adam / 0.0003
Label smoothing	0.1
Trainable parameters	10,106,688
Device	cpu

3.2 Training and Loss Curves

The submitted checkpoint was trained for 12 of 12 requested epochs (training_complete = True) on the 6,000-article CPU training subset, completing in 546.4 seconds (~9.1 minutes). Per-token cross-entropy loss (with label smoothing, padding ignored) fell from 7.0291 to 4.8451 on the training set and from 6.5491 to 5.7468 on the validation set, with the best validation loss (5.7468) used to select the checkpoint saved as best_model.pt.

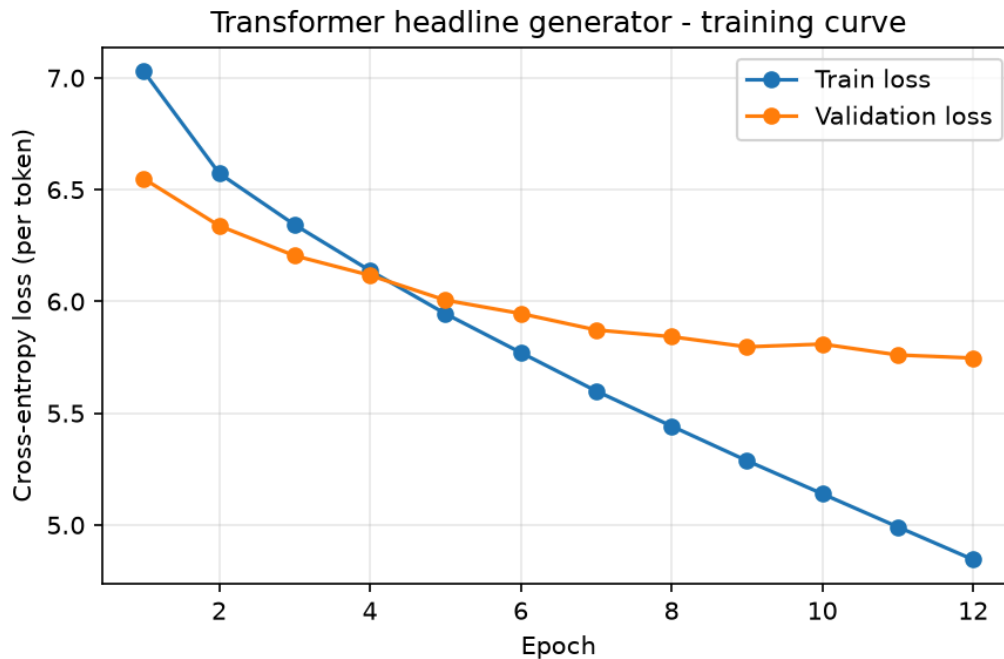


Figure 1: Training vs. validation loss curve (*models/transformer_scratch/loss_curve.png*)

3.3 Headline Generation (Decoding)

`training/infer.py` implements both decoding strategies used by the evaluation script and the web application:

- Greedy decoding: selects the highest-probability next token at each step, with n-gram repeat blocking (no immediate token repeat, no repeated 3-grams) to reduce stuttering.
- Beam-search decoding: maintains a configurable number of beams (1–8), applies length-penalised scoring, and returns up to top-k (1–5) distinct candidate headlines per article.

4. Application Development

The application (app/app.py) is a Flask web app that loads the trained checkpoint once per process and serves a single-page interface (app/templates/index.html).

4.1 Input Options

- Paste article text directly into a text box, or
- Upload a .txt file — a single article, or multiple articles separated by a blank line, or
- Upload a .csv file — one article per row, read from a column named article, text, body or content (first column used otherwise).
- Requests are capped at 10 MB and at most 25 articles are processed per request.

4.2 Output Display

- The top greedy-decoded headline is shown as the primary generated headline for each article.
- Up to top-k beam-search candidate headlines are listed underneath, when more than one is returned.
- Beam size and top-k are user-configurable form fields (beam size 1–8, top-k 1–5).

4.3 Screenshots from Virtual Lab:

Installation of required libs

```

# README.md
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

For a CPU-only PyTorch installation, install the remaining dependencies first,
then install PyTorch from its CPU wheel index:

'''bash
python -m pip install pandas numpy matplotlib Flask rouge_score nltk BeautifulSoup4 lxml
python -m pip install torch --index-url https://download.pytorch.org/whl/cpu
'''

Verify that the application dependencies and submitted model can be loaded:

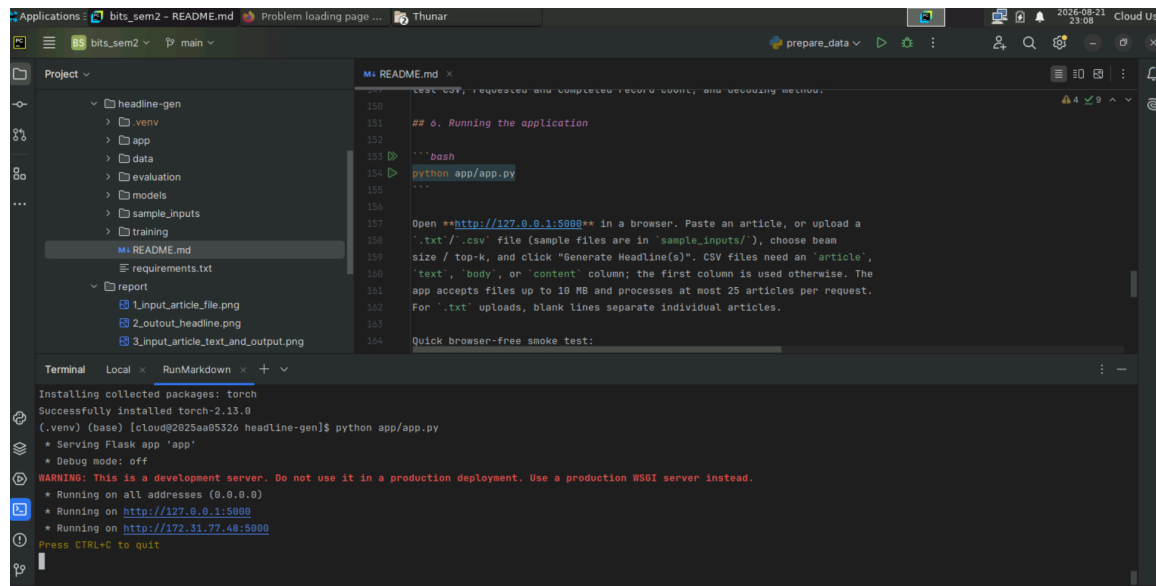
```

```

Terminal
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Downloading absl_py-2.5.0-py3-none-any.whl (137 kB)
Downloading defusedxml-0.7.1-py2.py3-none-any.whl (25 kB)
Downloading filelock-3.32.3-py3-none-any.whl (98 kB)
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading tqdm-4.70.0-py3-none-any.whl (80 kB)
Installing collected packages: nvidia-cusparselt-cu13, mpmath, cuda-toolkit, typing-extensions, triton, tqdm, sympy, soupsieve, six, setuptools, regex, pyparsing, pillow, packaging, nvidia-nvtx, nvidia-nvshmem-cu13, nvidia-nvjitlink, nvidia-nccl-cu13, nvidia-curand, nvidia-cufile, nvidia-cuda-runtime, nvidia-cuda-nvrtc, nvidia-cuda-cupti, numpy, networkx, markupsafe, lxml, kiwisolver, joblib, itsdangerous, fsspec, fonttools, filelock, defusedxml, cyclo, cuda-pathfinder, click, blinker, absl-py, werkzeug, python-dateutil, nvidia-cuspa, nvidia-cufft, nvidia-cublas, nltk, Jinja2, cuda-bindings, contourpy, BeautifulSoup4, rouge_score, pandas, nvidia-cusolver, nvidia-cudnn-cu13, matplotlib, Flask, torch
9/56 [setuptools]

```

Application Start up:



```
Project
├── headline-gen
│   ├── venv
│   ├── app
│   ├── data
│   ├── evaluation
│   ├── models
│   ├── sample_inputs
│   ├── training
│   └── report
├── requirements.txt
├── README.md
├── 1_input_article_file.png
├── 2_output_headline.png
└── 3_input_article_text_and_output.png
```

test CSV, requested and completed record count, and decoding method.

6. Running the application

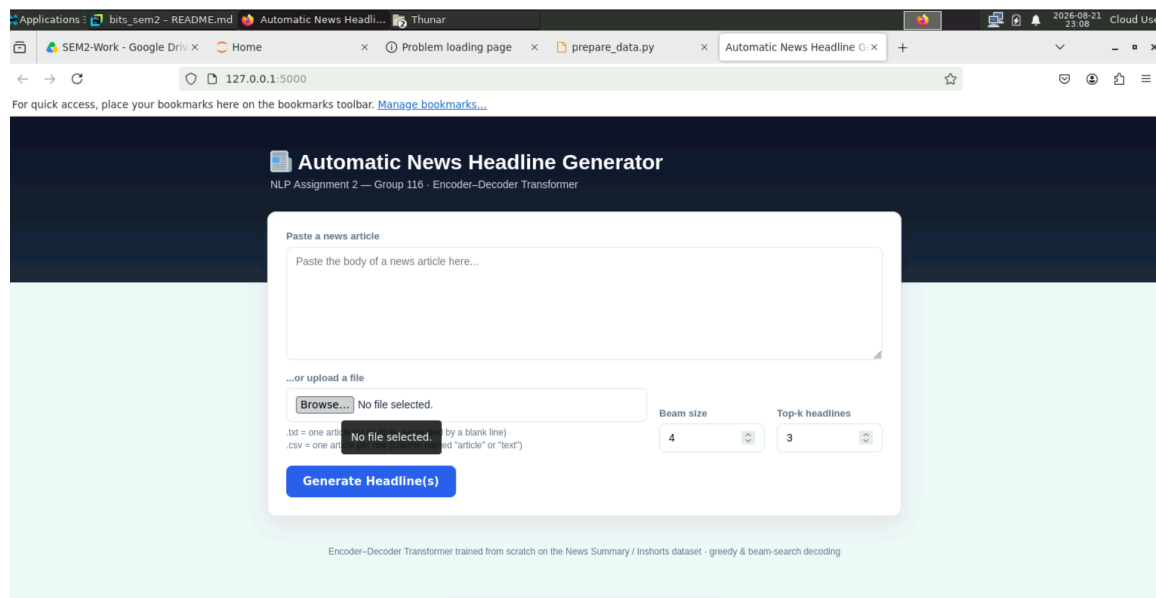
```
'''bash
python app/app.py
'''
```

Open **http://127.0.0.1:5000** in a browser. Paste an article, or upload a '.txt'/.csv' file (sample files are in 'sample_inputs/'), choose beam size / top-k, and click "Generate Headline(s)". CSV files need an 'article', 'text', 'body', or 'content' column; the first column is used otherwise. The app accepts files up to 10 MB and processes at most 25 articles per request. For '.txt' uploads, blank lines separate individual articles.

Quick browser-free smoke test:

Installing collected packages: torch
Successfully installed torch-2.13.0
(.venv) (base) [cloud@2025aa85326 headline-gen]\$ python app/app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.77.48:5000
Press CTRL+C to quit

Application Launch:



Application sample input:

The screenshot shows a web browser window with the URL `127.0.0.1:5000`. The page title is "Automatic News Headline Generator" and the subtitle is "NLP Assignment 2 — Group 116 · Encoder-Decoder Transformer". The main content area has a dark blue header with the title and subtitle. Below the header is a light blue box containing the input interface. The interface has a text area for pasting a news article, a file upload button, and a "Generate Headline(s)" button. The text area contains a news article about a woman jumping the White House fence. The file upload button is labeled "Browse..." and "No file selected.". The "Generate Headline(s)" button is blue and has the text "Generate Headline(s)".

Automatic News Headline Generator
NLP Assignment 2 — Group 116 · Encoder-Decoder Transformer

Paste a news article

Washington, Mar 27 (PTI) A 38-year-old woman in the US, who was apprehended twice for allegedly trying to jump the White House fence last week, has been arrested for scaling a fence at the Treasury Building. Marci Anderson Wahl of Everett, Washington, was arrested after an alarm sounded at about 2:15 am yesterday when she scaled a fence at the Treasury Building, next to the White House. Police said Wahl has told them she was there to speak to US President Donald Trump, the CNN reported. She was charged with unlawful entry and contempt of court. Wahl was first arrested on March 21 last week for trying to jump the White House fence. Once in custody, it was determined that Wahl had been issued a "stay away" order for the White House complex after the incident. On March 21, officers saw Wahl walking and staring at the White House complex before discovering she had jumped a fence on the south side but got stuck. Officers found her hanging from the inside of the fence by her shoelaces, which were "caught on top of the fence," according to a police report. She was re-arrested on March 24 after officers saw her near Lafayette Park. She was released on her own recognisance after a Saturday court appearance. In the incident, Wahl was charged with contempt of court in violation of a stay away order. She pleaded not guilty last Saturday and was again released on her own recognisance.

...or upload a file

Browse... No file selected.

Beam size 4 Top-k headlines 3

Generate Headline(s)

Application Out put:

The screenshot shows the same web browser window as the previous one, but now displaying the output of the application. The URL is `127.0.0.1:5000/generate`. The page title is "Automatic News Headline Generator" and the subtitle is "NLP Assignment 2 — Group 116 · Encoder-Decoder Transformer". The main content area has a dark blue header with the title and subtitle. Below the header is a light blue box containing the input interface. The text area contains the same news article as before. The file upload button is labeled "Browse..." and "No file selected.". The "Generate Headline(s)" button is blue and has the text "Generate Headline(s)".

Automatic News Headline Generator
NLP Assignment 2 — Group 116 · Encoder-Decoder Transformer

Paste a news article

Treasury Building, next to the White House. Police said Wahl has told them she was there to speak to US President Donald Trump, the CNN reported. She was charged with unlawful entry and contempt of court. Wahl was first arrested on March 21 last week for trying to jump the White House fence. Once in custody, it was determined that Wahl had been issued a "stay away" order for the White House complex after the incident. On March 21, officers saw Wahl walking and staring at the White House complex before discovering she had jumped a fence on the south side but got stuck. Officers found her hanging from the inside of the fence by her shoelaces, which were "caught on top of the fence," according to a police report. She was re-arrested on March 24 after officers saw her near Lafayette Park. She was released on her own recognisance after a Saturday court appearance. In the incident, Wahl was charged with contempt of court in violation of a stay away order. She pleaded not guilty last Saturday and was again released on her own recognisance.

...or upload a file

Browse... No file selected.

Beam size 4 Top-k headlines 3

Generate Headline(s)

Generated Headlines (1 article)

Input article:
Washington, Mar 27 (PTI) A 38-year-old woman in the US, who was apprehended twice for allegedly trying to jump the White House fence last week, has been arrested for scaling a fence at the Treasury Building. Marci Anderson Wahl of Everett, Washington, was arrested after an alarm ...

GENERATED HEADLINE

man arrested for at,

TOP-3 BEAM-SEARCH CANDIDATES

1. man arrested for
2. man arrested for raping woman
3. man arrested for raping woman in us

Encoder-Decoder Transformer trained from scratch on the News Summary / Inshorts dataset · greedy & beam-search decoding

Generated Headlines (1 article)**Input article:**

Washington, Mar 27 (PTI) A 38-year-old woman in the US, who was apprehended twice for allegedly trying to jump the White House fence last week, has been arrested for scaling a fence at the Treasury Building. Marci Anderson Wahl of Everett, Washington, was arrested after an alarm ...

GENERATED HEADLINE

man arrested for at,

TOP-3 BEAM-SEARCH CANDIDATES

1. man arrested for
2. man arrested for raping woman
3. man arrested for raping woman in us

Encoder-Decoder Transformer trained from scratch on the News Summary / Inshorts dataset · greedy & beam-search decoding

5. Evaluation and Demonstration

5.1 Quantitative Evaluation (ROUGE)

The final checkpoint was evaluated with `evaluation/evaluate.py` using greedy decoding over the complete held-out test split (800 of 800 records — not a partial sample), with ROUGE-1 / ROUGE-2 / ROUGE-L F1 computed via the `rouge_score` library (stemmer enabled).

Metric	F1 score (n = 800)
ROUGE-1	0.1472
ROUGE-2	0.0180
ROUGE-L	0.1364

These are modest scores rather than strong ones: they reflect a from-scratch, word-level Transformer trained on a 6,000-article subset for 12 CPU epochs, evaluated against exactly one reference headline per article (ROUGE penalises equally valid alternate phrasings). Full results, including per-example scores, are saved to `evaluation/results/rouge_scores.json` and `evaluation/results/sample_predictions.csv`.

5.2 Demonstration Across Different News Topics

The source dataset does not include an explicit category field, so four topically distinct held-out test examples were selected below to demonstrate the application across different kinds of news (sports, politics, entertainment and international/business), using the greedy-decoded output actually produced by the submitted checkpoint.

Category	Reference headline	Model output (greedy)	ROUGE-1 F1
Sports (tennis)	World no. 133 first mom to win WTA singles title in 6 years	's 1st ever world 's first in	0.30
Politics (defence)	Defence Minister to celebrate Diwali with jawans in Andaman	to launch with in india -	0.43
Entertainment (film)	From Sadma to Mom: Iconic roles of Sridevi in over 50 years	"film 'in india 's ':	0.13
International / business	Biggest hotel chain mistakes Hong Kong, Taiwan as countries	world 's largest world 'world 's world 'ships 's first time	0.00

Full article text for each example above is available in [evaluation/results/sample_predictions.csv](#).

5.3 Observations

Fluency and grammaticality:

Many generated headlines are ungrammatical fragments rather than well-formed sentences (e.g. "'s to be of: study", "world 's largest world 'world 's world 'ships 's first time"). This indicates the decoder has not yet learned reliable sentence structure at this training scale.

Repetition and generic wording:

Phrases such as "'s first ever", "world 's first" and "in the first time" recur across many unrelated articles regardless of topic. This is a generic/templated output pattern typical of small, lightly-trained decoder language models, rather than the model summarising each article's specific content.

Factual consistency:

In most examples, the generated headline does not reproduce the actual facts of the reference (e.g. the Marriott/Hong Kong/Taiwan example and the Neymar example both lose the actual subject matter entirely). A minority of examples (e.g. the Sitharaman/Andaman example, ROUGE-1 = 0.43) show the model can occasionally align a few correct keywords and the overall style with the reference.

Named-entity handling:

Proper nouns central to the story (YouTube, Shanghai, Marriott, Bondarenko, Neymar, Sridevi) are largely dropped or replaced with generic tokens. This is consistent with the word-level, 8,000-token vocabulary, under which rare or out-of-vocabulary named entities are mapped to <unk> at both training and inference time.

Likely causes and possible mitigations:

- Small training subset (6,000 of 88,535 available full-corpus records) and only 12 CPU epochs — training on the full split with a GPU is expected to improve fluency and content accuracy.
- Word-level tokenization — a sub-word tokenizer (e.g. byte-pair encoding) would reduce out-of-vocabulary named entities.
- Decoding — further tuning of beam size, length penalty and repeat-blocking may reduce generic/templated phrasing.

6. Conclusion

This project implements a complete, working, end-to-end abstractive headline generation pipeline: dataset selection and documented preprocessing, a from-scratch Transformer encoder–decoder trained with logged loss curves, greedy and beam-search decoding, a Flask web application supporting text and file-based input, and quantitative (ROUGE) plus qualitative evaluation on the complete held-out test split. Headline quality is modest, consistent with the CPU-only, subset-scale training used for this submission; the same code is designed to scale directly to the full ~88K-record corpus and more training epochs given GPU access, which is expected to substantially improve fluency, factual consistency and named-entity handling.

NLP-Assignment - 2 GS-2-Group 116-Completed: